

```
for rowIndex in range(BOARD_HEIGHT):
    # Check for four-in-a-row going across to the right:
    tile1 = board[(columnIndex, rowIndex)]
    tile2 = board[(columnIndex + 1, rowIndex)]
    tile3 = board[(columnIndex + 2, rowIndex)]
    tile4 = board[(columnIndex + 3, rowIndex)]
    if tile1 == tile2 == tile3 == tile4 == playerTile:
        return True
```

This function returns `True` if `playerTile` appears four times in a row horizontally, vertically, or diagonally. To figure out whether the condition is met, we have to check every set of four adjacent spaces on the board. We'll use a series of nested `for` loops to do this.

The `(columnIndex, rowIndex)` tuple represents a starting point. We check the starting point and the three spaces to the right of it for the `playerTile` string. If the starting space is `(columnIndex, rowIndex)`, the space to its right will be `(columnIndex + 1, rowIndex)`, and so on. We'll save the tiles in these four spaces to the variables `tile1`, `tile2`, `tile3`, and `tile4`. If all of these variables have the same value as `playerTile`, we've found a four-in-a-row, and the `isWinner()` function returns `True`.

In “Variables with Numeric Suffixes” on page 76, I mentioned that variable names with sequential numeric suffixes (like `tile1` through `tile4` in this game) are often a code smell that indicates you should use a single list instead. But in this context, these variable names are fine. We don't need to replace them with a list, because the Four-in-a-Row program will always require exactly four of these tile variables. Remember that a code smell doesn't necessarily indicate a problem; it only means we should take a second look and confirm that we've written our code in the most readable way. In this case, using a list would make our code more complicated, and it wouldn't add any benefit, so we'll stick to using `tile1`, `tile2`, `tile3`, and `tile4`.

We use a similar process to check for vertical four-in-a-row tiles:

```
for columnIndex in range(BOARD_WIDTH):
    for rowIndex in range(BOARD_HEIGHT - 3):
        # Check for four-in-a-row going down:
        tile1 = board[(columnIndex, rowIndex)]
        tile2 = board[(columnIndex, rowIndex + 1)]
        tile3 = board[(columnIndex, rowIndex + 2)]
```